基于 PYNQ Z2 与 RT1064 的智能车视觉交互与运动控制系统

摘要

本系统实现了麦克纳姆轮小车的全向运动、视觉交互、近区障碍绕行和轨迹显示。按键与屏幕菜单设置路线、距离、速度及转角，支持分步和连续运行。颜色卡模拟红停绿行、黄灯等待，0～9 数字卡设置下一段平移速度；镜头前方约 50 cm 通道内确认障碍后，小车停稳并按右移、前进、左移的顺序绕行，再继续原步骤。屏幕同步显示识别结果、运动状态和推算轨迹，结束后保存报告。硬件采用自绘主板、驱动板及 3D 打印支架。

第一部分　作品概述

1.1 功能与特性　支持全向平移、原地转向及多段路线，集成按键选参、颜色启停、数字单段调速、近区障碍判断与自动绕行、轨迹显示和报告保存。

1.2 选题与应用　选择自主选题，面向智能车竞赛演示、运动控制调试和 FPGA 软硬件协同实验。

1.3 主要技术特点　RT1064 执行电机闭环，PL 完成编码器与陀螺数据运算，ARM 处理视觉、路线、界面和报告。

1.4 主要参数

项目	设置范围
菜单平移距离	2～4 m，步长 0.2 m，循环选择
分步转向角度	90°、180°、360°、720°
菜单速度上限	20%～200%，步长 20%
数字单段速度	0～9 对应基准速度的 60%～150%

1.5 主要创新点　将视觉交互与路线执行联动：颜色控制通行，数字调整单段速度，近区障碍触发停稳绕行；预计路线、推算轨迹和报告共同呈现运行过程。

1.6 设计流程　装配 → 控制 → 识别 → 绕行 → 实车验证。

第二部分　系统组成及功能说明

2.1 总体方案

RT1064 执行路线和电机控制，并通过 UART 向 PYNQ 发送编码器、IMU 和运动状态。PYNQ 将原始数据交给 PL 运算，得到速度、路程、航向和车体位移；ARM 组合平面坐标，完成识别、轨迹绘制和报告保存，再通过 HDMI 输出画面。

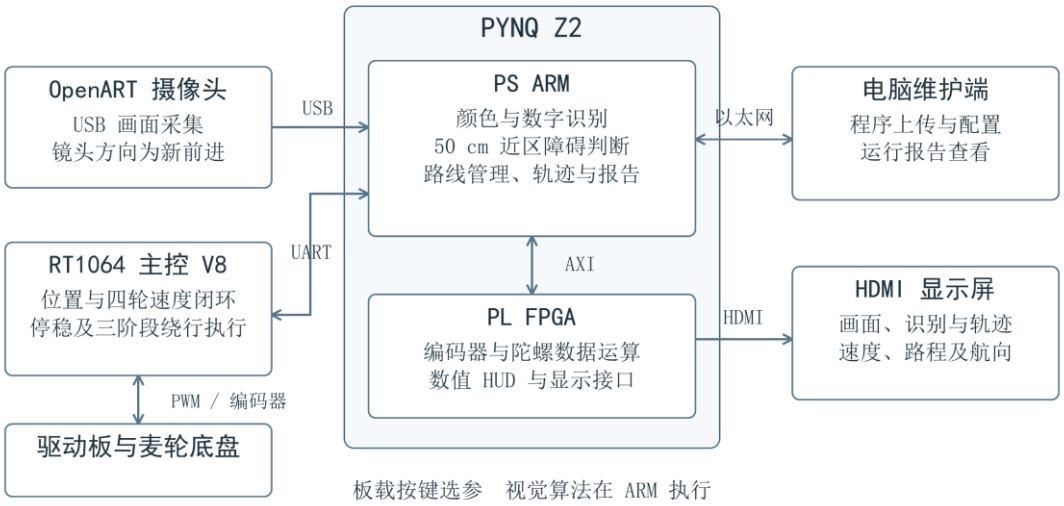


图 1　系统结构及软硬件分工

2.2 硬件组成

小车采用四轮麦克纳姆底盘，RT1064 和驱动板位于底盘层，PYNQ 与显示屏由 3D 打印支架承载。摄像头物理安装未变，仍朝原车尾；该方向已定义为新的前进方向，原车尾成为新车头。电脑用于上传程序和查看报告，板端可独立运行。

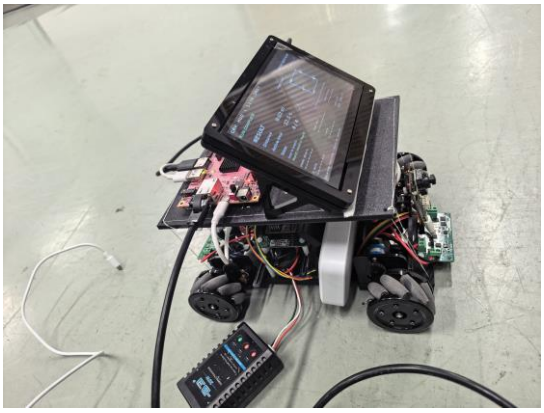


图 2　整车装配实拍，保留原安装结构

2.2.1 3D 打印安装支架

3D 打印支架用于固定 PYNQ Z2 与显示屏。安装板为 220 mm×180 mm，总高 65 mm、顶板厚 5 mm；工程图、STL 模型及打印实物如下。

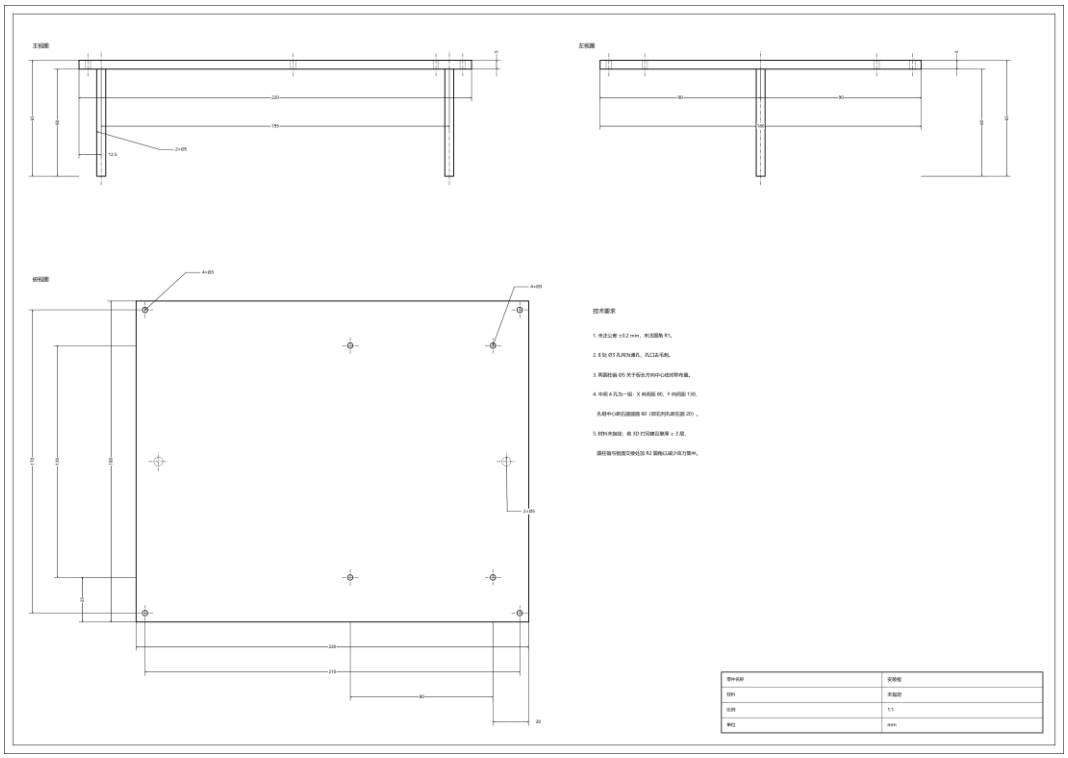
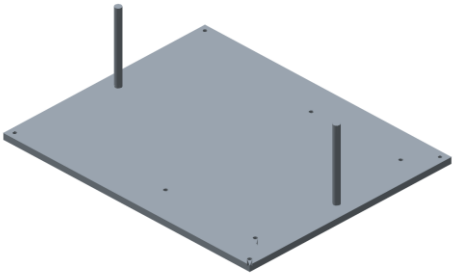


图 3　安装支架结构工程图，尺寸单位为 mm



STL 外形：220 × 180 × 65 mm



图 4　左：打印文件 STL 模型预览；右：3D 打印实物

2.2.2 自主电路设计与接线

主板与驱动板均由团队自行设计绘制。主板用嘉立创 EDA 完成原理图与 PCB，提供 3.3 V、5 V 供电及编码器、IMU 等接口。驱动板驱动四轮电机；板间 UART 为 115200 bit/s、8N1。

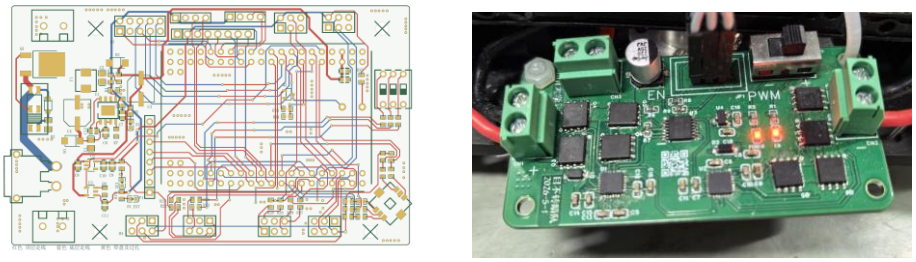


图 5 左为主板 PCB 布线图 右为自绘驱动板实物

主板图依据原始工程数据绘制，显示走线、焊盘与过孔。

车板接口	PYNQ 接口	信号
H10 4 TX	PMOD B 1 RX	遥测与运动状态
H10 3 RX	PMOD B 2 TX	路线及控制命令
H10 2 GND	PMOD B 5 GND	共地

UART 为 3.3 V TTL；车板 H10 1 的 5 V 不接入 PYNQ IO。

2.3 软件实现

采集与校准 RT1064 每 20 ms 累计修正后的四轮编码器，每 5 ms 累计陀螺 Z 轴值，组成 RAW33 帧。PYNQ 检查 CRC、序号及计时，静止约 3 s 估计零偏。

PL 计算 四轮计数差换算横、纵位移，位移模除以采样间隔得到速度，累加得到路程；扣除零偏的陀螺值按采样时间积分，得到相对航向。

编码器与 IMU 融合 编码器提供位移，陀螺仪提供方向。ARM 用相邻航向的中间角 θm 旋转车体增量 Δxb、Δyb，再累计为固定坐标系的 X/Y。

$$x_k = x_{k-1} + cos\theta_m \cdot \Delta x_b - sin\theta_m \cdot \Delta y_b$$
$$y_k = y_{k-1} + sin\theta_m \cdot \Delta x_b + cos\theta_m \cdot \Delta y_b$$

b 为车体坐标，k 为更新序号。车板 EKF 校正横滚、俯仰，平面航向用陀螺积分。应用层将原固定平面坐标旋转 180°，使镜头前进为 +Y、镜头右侧为 +X；航向保持相对静止校准值。

2.3.1 按键选参与屏幕 UI

运行参数可直接在 PYNQ 板载按键和 HDMI 屏幕上选择。小车停在待机或任务结束状态时，短按 BTN2 打开 RUN SETTINGS；高亮条指示当前选项，右侧显示参数值，下方保留按键提示。菜单打开期间不发起运动。



图 6　左：参数选择实拍（60%）；右：屏幕运行界面实拍

按键	参数选择界面中的作用
BTN0	移动高亮条，选择下一个参数
BTN1	循环切换当前选项或增加参数值
BTN2	保存设置并退出菜单
BTN3	取消本次修改并退出菜单

可选项目包括 MODE、ROUTE、LENGTH、WIDTH、SPEED LIMIT 和 TURN ANGLE。分步模式可选两圈、矩形、往返、单次平移和单次旋转；连续模式可选前三种路线。矩形可分别设长、宽，单次旋转显示 TURN SPEED；分步两圈与旋转可选 90°、180°、360°、720°，连续两圈固定为 90°。距离与速度达到上限后循环回到起始值。

操作示例　短按 BTN2 进入菜单 → 用 BTN0 选中 SPEED LIMIT → 按 BTN1 调整速度上限 → 按 BTN2 保存 → 回到待机画面后按 BTN0 发车。保存参数不会自动启动。

运行界面　左侧显示摄像头与路线提示，右侧为轨迹小地图；下方显示速度、累计路程、航向、X/Y 位置和 UART 状态。任务结束后切换到结果页，保留轨迹及本轮统计。

2.3.2 图像采集与颜色识别

OpenART 摄像头通过 USB 向 PYNQ 提供画面。ARM 用 OpenCV 将图像转为 HSV，分别提取红、绿、蓝、黄区域，再用 3×3 闭运算连接小缺口，按面积、长宽比、填充率和边界位置筛选，排除零散噪点及大块背景。下图为实际采集样本。




图 7　红、绿目标采集画面，对应输出 RED、GREEN

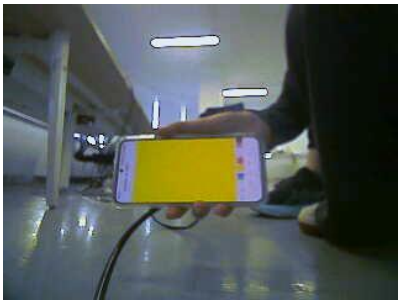



图 8　蓝、黄目标采集画面，对应输出 BLUE、YELLOW

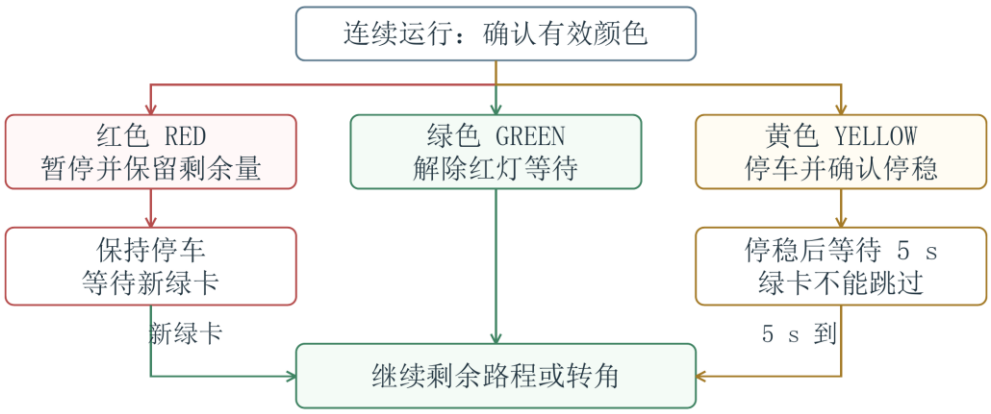


图 9　连续模式下模拟红绿灯通行的处理流程

控制前要求两次新鲜观测，结果超过 0.5 s 或已切换步骤便丢弃。连续模式同画面多色时按红、黄、绿优先级处理；黄灯等待 5 s 期间绿卡不能提前放行，出现红卡则等待新绿卡。蓝色仅显示和记录。同卡持续出示只触发一次，移开约半秒后可再触发。分步停稳后用绿卡继续；首次及新一轮由 BTN0 启动。

本次静态采样有效帧：红 5/5、绿 4/4、蓝 4/4、黄 5/5；绿、蓝各排除一帧切换画面。

2.3.3 手写数字识别与单段调速

数字卡采用白底黑字，识别单个 0～9。ARM 提取并预处理数字区域，采用公开预训练 MNIST ONNX 模型[1]，通过 OpenCV DNN[2] 推理，将识别结果用于下一段平移调速。

处理过程 灰度化与自适应阈值 → 剔除彩色区域 → 提取数字轮廓 → 长边缩放到 20 像素 → 按笔画重心置于 28×28 黑底图 → 归一化并推理。

结果确认 最高类别分数不低于 0.70，且领先第二类别至少 0.25；连续两帧类别一致才确认。



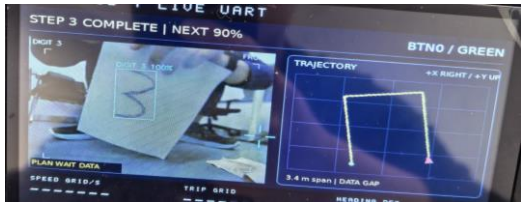


图 10 数字识别实拍：左 2 → NEXT 80%；右 3 → NEXT 90%



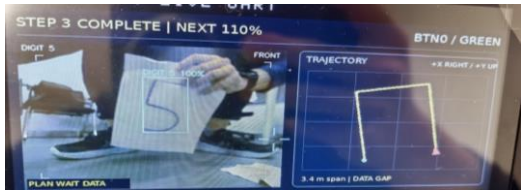


图 11 数字识别实拍：左 4 → NEXT 100%；右 5 → NEXT 110%

画面框出数字并标注类别及模型得分，顶部 NEXT 显示下一段平移的速度上限。识别框中的百分比表示模型得分，NEXT 百分比表示速度设置。

数字如何影响小车

数字	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
速度上限	60%	70%	80%	90%	100%	110%	120%	130%	140%	150%

数字设置只覆盖下一段平移，不与菜单速度相乘。例如菜单设为 80%，识别“6”后下一段用 120%，完成后恢复 80%；若下一步是转向，则保留到下一段平移。

运行条件 分步模式停稳暂停时识别数字，连续模式只检测颜色。旧识别结果不参与控制，识别事件在小地图中用 V 标出。

2.3.4 近区障碍识别与倒影过滤

障碍判断集中在镜头正前方约 50 cm 内的中央通道。静止时用三帧稳定画面学习通道外地面的外观，运行时保留地面参考，从物体与地面的颜色、亮度差提取候选区域。画面显示黄色 50 cm 边界和近区通道，标线在识别后绘制。



图 12　静态采图检测叠加：左为 50 cm 杯子，右为约 1 m 对照

边界标定　以摄像头地面垂足为起点，量取 50 cm 放置杯子。在 8 张 320×240 原图中，杯底接触边缘均位于第 179 行，据此设置近区边界。以物体估计落地点是否进入通道判定，物体上部跨线不触发。

倒影处理　寻找实体到地面的较强边缘作为接触点，排除较弱的倒影拖尾，避免检测框沿反射向下延长。地面学习避开中央通道，运行期间不重学前方障碍。

静态采图场景	图像数	检测结果
50 cm 杯子	8	8 张检出 BLOCKED
约 1 m 杯子及倒影	8	8 张 CLEAR
空地	5	5 张 CLEAR

该边界用于固定相机、平地条件下的近区演示，属于单距离标定；强倒影或地面变化仍可能影响接触点判断。

2.3.5 停稳后的自动绕行

前进步骤中连续两帧确认近区障碍后，PYNQ 请求停车；RT1064 确认停稳后自动执行右移 0.5 m、沿原方向前进 1 m、左移 0.5 m，保持朝向并继续原步骤剩余目标，普通绕行开始前无需再按 BTN0。

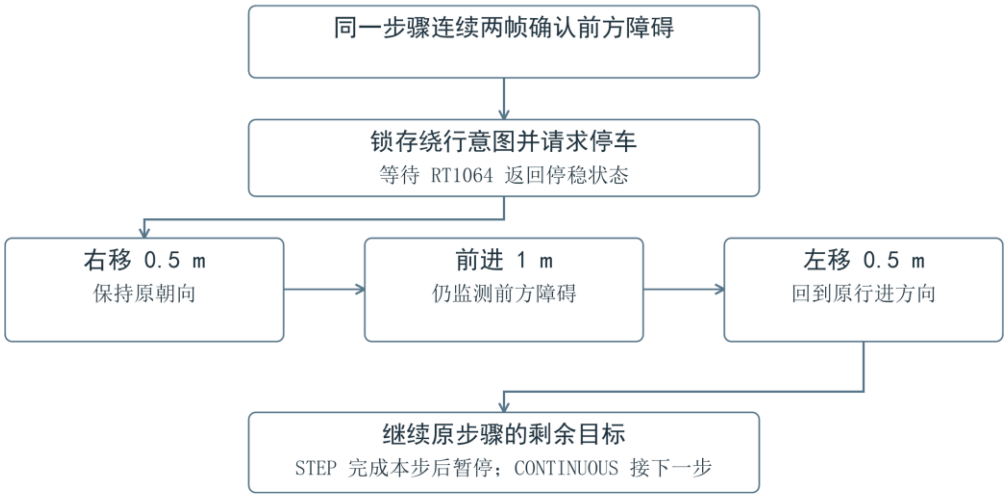


图 13 已确认障碍触发停稳与自动绕行的流程

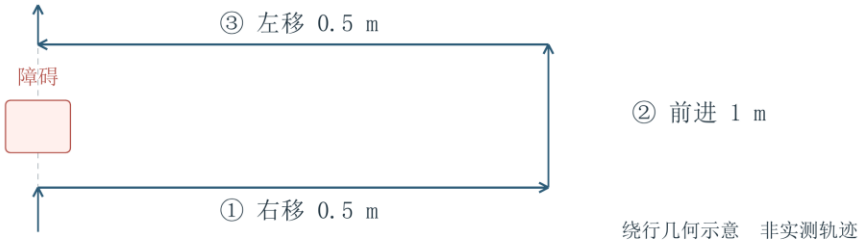


图 14 右移、前进、左移的指令路径示意

意图锁存 两帧确认后，绕行意图绑定当前会话、步骤与定位参考。刹车时短暂 CLEAR 或 UNKNOWN 不再丢失已确认请求；切换步骤、定位参考、新运行或 BTN1 结束时清除。红黄灯等待优先保留，解除后再执行已确认绕行。

适用条件 V8 固定绕行要求本步骤剩余不少于 1 m。侧向通道按本次演示要求视为空闲；前进绕行段仍检测前方，新障碍出现时停车，不嵌套绕行。连续三帧 CLEAR 后由 BTN0 或绿色恢复。

绕行及原步骤完成后，STEP 暂停等待下一步，CONTINUOUS 继续后续步骤；BTN1 仍结束整轮。

2.3.6 轨迹显示与运行记录

小地图显示编码器与 IMU 推算的行驶轨迹，以本轮起点为参考，+X 向右、+Y 向上，比例随范围自动调整。三角形表示车体朝向，T 标记转向结束位置，V 标记识别发生时的车辆位置。摄像头中的预计路线用于提示，小地图记录行驶过程。

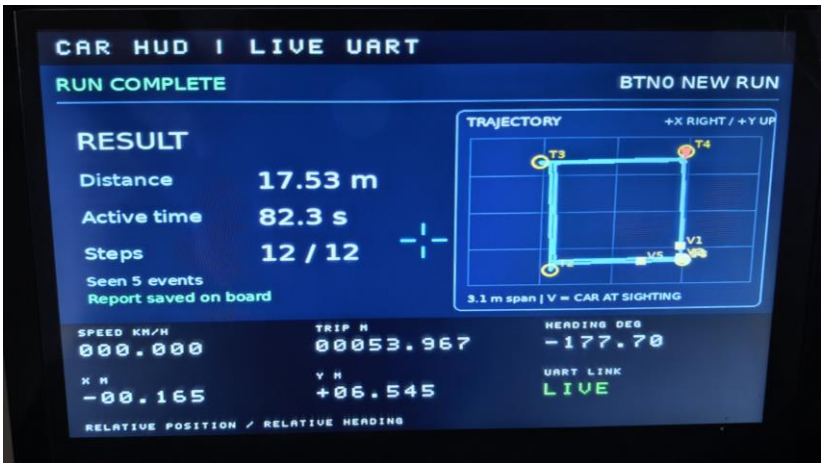


图 15　2026 年 10 月十二动作运行结果与轨迹实拍

实拍为 12/12 步、17.53 m、动作用时 82.3 s、5 次识别事件。本轮统计以左侧 RESULT 为准，底部 TRIP 为遥测显示值。结束后保留轨迹，并保存 CSV、JSON 和 HTML 报告。

当前版本以校准时镜头前进方向建立固定坐标，并以本轮起点绘制相对轨迹。ARM 对平面 X/Y 作 180° 坐标转换，路线命令同步转换到车板坐标；速度模和累计路程不变。

镜头画面叠加预计路线，小地图保留实际推算轨迹。绕行期间暂不叠加原路线提示，防止将绕行阶段误画为原步骤；识别、转向和运行事件写入报告。

第三部分　完成情况及性能参数

本次在原全向运动、按键菜单、颜色与数字交互、轨迹和报告基础上，增加 50 cm 近区边界、倒影过滤及停稳自动绕行。2026 年 11 月 1 日，用户反馈“验证无误”，作为本轮功能验收记录。

3.1 最新静态记录与软件验证

检查内容	**2026 年 11 月 1 日记录**
摄像头静态采集	60.093 s；269 帧；平均 4.476 FPS
有效遥测	601 帧；CRC 错误、丢帧、串口 overrun 均为 0
应用测试	离线 292 项、PYNQ 板端 292 项均通过
Python 与实际 C 控制代码联调	8 项理想模型检查通过
旧 V7 协议联调	11 项通过
部署与失败回滚模拟	5 项通过

近远杯子实时状态各读取 20 次：50 cm 为 19 次 BLOCKED、1 次 UNKNOWN，覆盖 5 个不同快照；约 1 m 为 15 次 CLEAR、5 次 UNKNOWN，覆盖 6 个不同快照。状态读取、静态采图和软件模型检查分别统计。

3.2 发布与运行资料

当前配套为 PYNQ 发布源码与 RT1064 V8 车板程序，板端发布清单包含 120 个文件。原始采图、测试日志和部署记录用于核对本报告；仓库按自主选题要求整理最终工程及精选验证材料。

发布清单 SHA256：
e6ce39d683bf13402295bf3298508e889c8dd72d13653267e14333c1b54d3851

项目仓库：https://github.com/gtrczxseu-web/pynq-z2-rt1064-smartcar

上述帧率取自本轮静态采集；历史十二动作照片与下一节卷尺测量保留原记录。

3.3 麦克纳姆轮底盘定距实测

2026 年 10 月，在 STEP、COMPETITION 模式下设置 2.00 m 目标，分别选择四个速度上限，完成首段直线运动后用卷尺核对距离。

速度上限	目标 / m	实测约 / m	距离偏差	相对误差
60%	2.00	2.00	约 0 cm	约 0%
100%	2.00	2.03	+3 cm	约 1.5%
200%	2.00	2.12	+12 cm	约 6.0%
140%	2.00	2.07	+7 cm	约 3.5%

距离偏差＝实测距离－目标距离；相对误差＝（偏差绝对值÷2.00 m）×100%。速度百分比表示控制上限。

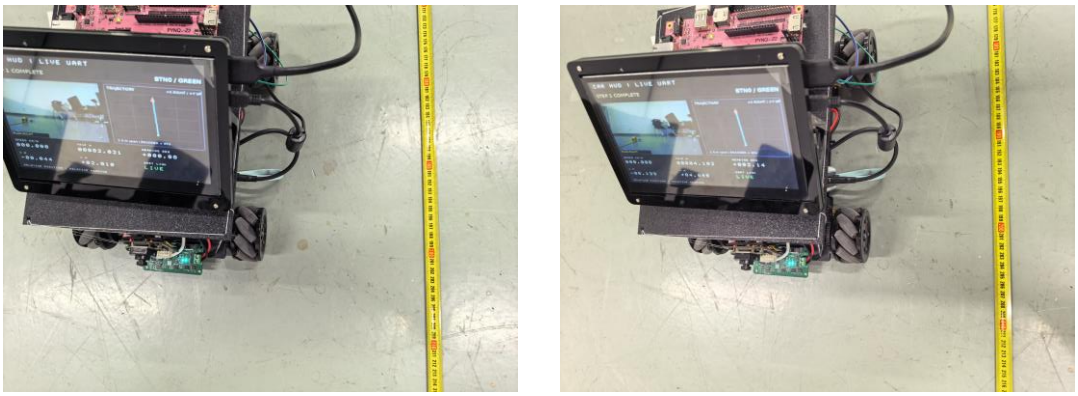


图 16　左：60% 约 2.00 m；右：100% 约 2.03 m

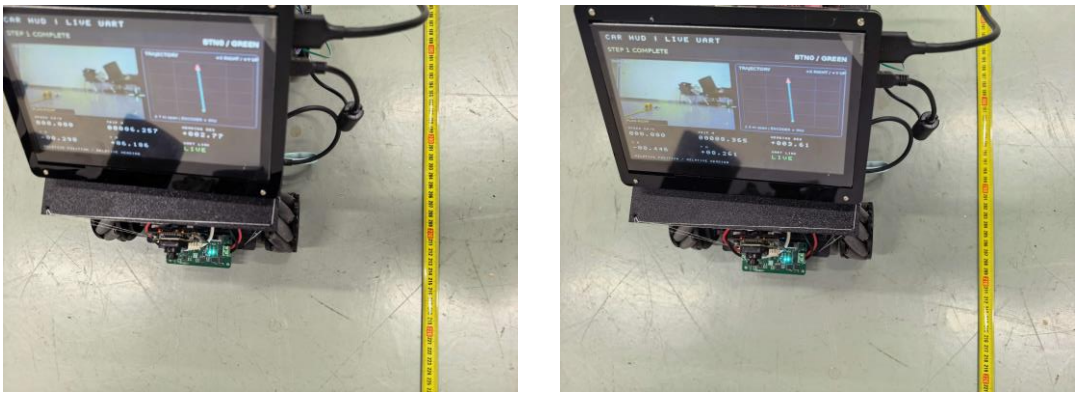


图 17　左：200% 约 2.12 m；右：140% 约 2.07 m

第四部分　设计依据与优化过程

4.1 选题背景与实现重点

本项目将视觉识别结果用于小车控制，并在屏幕上显示执行过程。颜色决定通行与等待，数字设置下一段平移速度，近区障碍触发停车及绕行。运行状态和轨迹与摄像头画面同屏显示，便于现场调试，也便于核对识别事件是否作用于正确的步骤。

图像处理采用 OpenCV，数字推理使用公开 MNIST 模型。团队完成主板和驱动板设计、车板闭环与路线控制、板间通信、传感数据处理、视觉事件控制及屏幕界面集成。项目重点是这些功能在同一运行流程中的协作。

4.2 软硬件划分依据与接口

执行位置	承担任务	划分依据与接口
RT1064	四轮闭环与动作执行	固定周期采集；UART 上传 RAW33 和状态，接收路线与控制命令
PYNQ PL	位移、路程、速度、航向计算及显示链路	重复运算由寄存器接口调用；ARM 提交计数并读取结果
PYNQ ARM	识别、状态管理、坐标累计和 UI	适合图像处理及流程调整；USB 采图，HDMI 输出

PL 驱动核对 SP21 标识 0x53503231，写入 0x40 起输入寄存器，以 0x60 发起计算；0x68/0x6C 为有效/拒绝计数，0x70～0x84 为运动结果。数字模型运行在 ARM。

4.3 从故障到修复

自动绕行初版在停车后再次要求 BLOCKED。刹车时画面短暂变为 CLEAR，就丢失绕行请求。修复后，已确认意图绑定会话、步骤和定位参考，停稳后执行；切换上下文或停止时清除，红黄灯仍优先。

同组检查	修复前	修复后
绕行针对性用例 17 项	12 通过，5 失败	17 项通过
新增回归覆盖	—	增加 6 项；应用总计 292 项通过

对比来自 before_fix_tests.log 与 application_tests.log，反映控制逻辑改进。FPGA 资源、时钟与时序优化前后数据尚未取得对应报告，未填入推算值。

第五部分　大模型协作与技能提炼

5.1 协作记录

团队通过 Codex 辅助检查代码、定位故障、补充回归检查和整理文档；车辆操作、现象反馈及实物照片由团队提供。以下保留本次故障处理中的提示与修正要点，模型输出以可复核的代码和日志确认。

提示一　“现在为什么不动了”

模型读取状态后发现 OBSTACLE WAIT，前方已是 CLEAR，回答可用 BTN0 或绿色恢复。该回答解释了当前等待状态，但没有解决“应当自动绕行”的问题。

提示二　“识别到了障碍不应该绕行吗”

模型随后自查停车与绕行衔接，指出停稳后重新检查 BLOCKED 会丢失已确认请求，改为锁存意图。先补充停车中变为 CLEAR、UNKNOWN，以及步骤和定位参考切换等用例，再修改实现。

验证反馈　修复前针对性检查为 17 项、5 项失败；修复后应用检查扩展至 292 项，离线与板端均通过。模型向团队说明无需重新烧录车板，并建议在 60% 速度、约 50 cm 杯子条件下复测；团队随后反馈“验证无误”。

以上为 2026 年 11 月 1 日协作摘录及响应摘要；对应源码为 motion_link.py、standalone.py，原日志保留失败记录和部署回执。未以团队反馈推导新的定量实测指标。

5.2 提炼为可复用 Skill

从该问题提炼“异步视觉事件与动作确认分离”的处理方法：视觉端负责确认事件，控制端等待停稳后消费事件；事件必须绑定上下文，不能被后续一帧抖动覆盖，也不能跨步骤复用。

使用步骤　记录事件与动作时序 → 构造触发后状态变化的回放 → 绑定会话、步骤和定位参考 → 检查停止与红黄灯优先级 → 对照修复前后日志。

适用范围　适用于识别与执行异步、动作有确认延迟的控制流程。用例验证限于本项目；更换协议、设备或路线时，需要重新核对状态和上下文。

技能包内容　在 skill/README.md 中说明输入、流程、失效条件与验证方法，关联针对性用例和修复前后结果；协作记录保留提示、响应及自查过程。

第六部分　复现步骤与验收方法

6.1 环境与版本

硬件采用 PYNQ Z2、RT1064、OpenART 摄像头及 HDMI 屏幕。PYNQ 依赖 Python 3、PYNQ 库、NumPy 和含 DNN 的 OpenCV；车板使用配套 Keil 工程，模型使用随包版本。

hud_live.hwh 标注 Vivado 2018.3，器件为 7z020、clg400、-1。指南推荐 2026.1 或 2025.2；现有硬件描述不能代替完整工程及工具重建记录。

6.2 先完成离线检查

步骤 1　获取文件　取得同版 PYNQ 程序、配置、模型、bit/HWH 对及 RT1064 工程，按清单核对文件和哈希。目录改名时同步修改工程引用。

步骤 2　回放软件检查　在 PYNQ 程序目录执行 python3 -m unittest discover -s verification -p "test_*.py"，核对绕行用例与应用结果。软件检查与实车验收分别记录。

步骤 3　构建车板　打开 rt1064.uvprojx，使用配套依赖，核对芯片、编译器和目标配置后构建。验收版本 AXF 的 SHA256 前 16 位为 56ad2df2484af36f。

6.3 板端复现与功能验收

步骤 4　接线与部署　按第 2.2.2 节接线，使用配套 V8 车板程序。电机驱动关闭时核对 bit/HWH、SP21 标识、遥测和摄像头，保留原版本以便回退。

步骤 5　启动与校准　在板端发布目录运行 sudo python3 standalone.py --config autostart.json，或使用配套服务。静止约 3 s 完成校准，核对 READY、UART LIVE 和画面。

步骤 6　逐项验收　确认场地后由操作者开启驱动，从 60% 开始，检查按键、颜色通行、数字调速、近区绕行和 BTN1 结束，核对轨迹及 CSV、JSON、HTML 报告。固定绕行要求本步剩余不少于 1 m，侧向通道空闲。

以上为交付运行版本的复现路径；PL 从源码到 bitstream 的重建仍需第七部分所列材料。

第七部分　自主选题材料对应表

本报告按指南 3.3 自主选题编写。报告覆盖背景与创新点、设计原理及框图、软硬件划分与接口、优化过程、大模型协作、Skill 提炼和复现步骤；工程附件应与报告中的版本及结果对应。

指南要求	本项目材料与状态
功能与板上运行	第二、三部分；实拍、原始采图、板端检查及运行记录
PS 与 MCU 源码	PYNQ Python/C 程序、模型与配置；RT1064 Keil 工程及依赖
验证与参考结果	应用日志、Python/C 联调、部署回滚模拟、50 cm 原始采图及定距实测
大模型协作与 Skill	第五部分记录方法与案例；附件需保留提示响应摘录及 skill/README.md
完整 FPGA 工程	已有 bit/HWH；对应 RTL/HLS、BD 重建脚本、XDC 等完整重建材料待补齐
综合实现与性能资源	对应资源利用率、时钟与时序报告待补齐；当前不提供虚构数据

提交组织与许可证

提交目录及文件名使用英文，避免空格和特殊字符；源码、验证、构建、板端材料、数据、Skill 与报告可分别放入 src、sim、build、board、data、skill、report。目录为指南推荐结构，工程内引用必须随整理同步更新。

第三方代码与模型保留原许可证，逐飞库涉及 GPLv3，MNIST 模型为 MIT；不能将整个项目统一标成 MIT。仓库链接见第 3.2 节，公开性和上传完整性以提交时实际状态为准。

依据：AMD 2026 FPGA 赛道选题指南 3.3.3、3.3.5，正文第 20～22 页。进入自主选题决赛后，另按模板准备一页英文 A4 海报。

参考文献

[1] ONNX. MNIST Handwritten Digit Classification Model[EB/OL]. [2026-10-28]. https://github.com/onnx/models/tree/main/validated/vision/classification/mnist.

[2] OpenCV. Deep Neural Network Module[EB/OL]. [2026-10-28]. https://docs.opencv.org/4.x/d6/d0f/group__dnn.html.